

Lecture 3-1 - Loops and invariants

Loop invariants are assertions essential for verifying loop properties. In this lecture, we illustrate this concept and show how invariants can be used in the construction of loops. Secondly, we treat functions both as specification tools and as components of Dafny code (since version 4.0). Because functions and methods may be recursive, we then face the problem of proving their termination.

Loop invariants

Control structures for loops, such as while commands in IMP, are necessary in practical programming and unavoidable in any Turing-complete programming language, as is the case with IMP. However, reasoning backward with wp-predicate transformers is no longer effective when considering loops because of two issues. First, the truth of the predicate $\text{wp}(\text{while } b \text{ do } S, Q)$ in at least one state implies termination in that state, which cannot be guaranteed in general and is undecidable in principle.

Second, given that we know that the execution of while b do S, starting from the given state, will end after k consecutive executions of S, and that all of these executions will properly terminate, the formula that we obtain from the equation concerning sequential composition (item 2 of the wp-properties in Lecture 2) would be:

$$\text{wp}(S, \text{wp}(S, \dots \text{wp}(S, Q))) \quad \text{with } k \text{ times nesting of wp's}$$

This will represent a sort of symbolic trace of the executions that is hardly suggestive of meaningful properties of the loop itself or of the idea behind the definition of the specific while command.

As a first step toward resolving the difficulty, we shall consider the weaker property expressed by the notion of "weakest liberal precondition" and the respective predicate transformers, which are

$$\text{wlp}(S, Q) \ s \iff \text{forall } t. (S, s) \Rightarrow t \implies Q \ s$$

If the execution of S in s does not terminate, then the implication above is vacuously true. On the other hand, it is easy to see that $\text{wp}(S, \text{true}) \ s$ is true if and only if the execution of S from s terminates properly. Hence

$$\text{wp}(S, Q) \ s \iff \text{wlp}(S, Q) \ s \ \&\& \ \text{wp}(S, \text{true}) \ s$$

so that using wlp we can factor away the issue of termination, which is treated in most of the

proof assistants, like Dafny, through decreasing clauses w.r.t. some well-founded ordering (see the next lecture).

With wlp we can prove the following:

Theorem (soundness and - relative - completeness of Hoare Logic) For all predicates P , Q , and statement S of IMP:

$\{P\} S \{Q\}$ is derivable in Hoare logic if and only if $P \implies \text{wlp}(S, Q)$.

In particular, the rule for the while command in the logic is

$$\text{Loop} \frac{\{P \ \&\& \ b\} S \{P\}}{\{P\} \text{ while } b \text{ do } S \{P \ \&\& \ !b\}}$$

The predicate P is called a **loop invariant**: in the premise $\{P \ \&\& \ b\} S \{P\}$, we suppose that the truth of P will be preserved by any execution of S in a state where the "guard" b is true.

Consequently, if P is true in the initial state of the execution of `while b do S`, then if and when this terminates, P will still hold in the final state, no matter how many times the "loop body" S has been executed, together with $!b$, which is the exit clause of the loop.

The same rule in (pseudo) Dafny has the form:

```
{{ P }}
while B
  invariant J
{
  Body
}
{{ P && !B }}
```

To prove this triple Dafny will try:

```
{{ P && B }}
Body
{{ P }}
```

Note. The double curly braces $\{\{...\}\}$ are not Dafny syntax; also, this rule omits the termination clause; for a full formulation, see lecture 4. Such a clause is automatically inferred in some easy cases, where it can be omitted.

The method `LoopSpecifications()` includes three loops without a body; nevertheless, Dafny checks whether the post-condition assertion follows from the respective invariants.

```

method LoopSpecifications()
{
    var x := 0;
    while x < 300
        invariant x % 2 == 0
        // here the body is omitted
    assert x % 2 == 0 && 300 <= x;

    var y := 2;
    while y < 50
        invariant y % 2 == 0
    // assert .... (write here the post-condition)

    var z := 0;
    while z % 2 == 0
        invariant 0 <= z <= 20
    // assert z == 19; (not provable: why?)
}

```

The next examples illustrate the use of invariants as specifications of loops whose body consists of just an increment or a decrement of a variable.

```

method UpWhileLess(N: int) returns (i: int)
    requires 0 <= N
    ensures i == N
{
    i := 0;
    while i < N
        invariant i <= N
    {
        i := i + 1;
    }
}

```

and

```

method UpWhileNotEqual(N: int) returns (i: int)
    requires 0 <= N
    ensures i == N
{
    i := 0;
    while i != N
        invariant i <= N
    {
        i := i + 1;
    }
}

```

and

```
method DownWhileNotEqual(N: int) returns (i: int)
  requires 0 <= N
  ensures i == 0
{
  i := N;
  while i != 0
    invariant 0 <= i
    {
      i := i - 1;
    }
}
```

and

```
method DownWhileGreater(N: int) returns (i: int)
  requires 0 <= N
  ensures i == 0
{
  i := N;
  while 0 < i
    invariant 0 <= i
    {
      i := i - 1;
    }
}
```

The final example verifies an iterative implementation of the Gauss formula:

$$1 + 2 + \dots + n = n * (n + 1) / 2$$

that is

```
method sum(n: nat) returns (res: nat)
  ensures res == n * (n + 1) / 2
{
  res := 0;
  var i := 0;
  while i <= n
    invariant i <= (n + 1)
    invariant res == (i - 1) * i / 2
    {
      res := res + i;
      i := i + 1;
    }
}
```

To demonstrate how the method `sum()` has been verified (at least in principle), look at the following commented code, where the construct `calc { ... }` enforces Dafny to check the equations inside the curly braces.

```
method sum(n: nat) returns (res: nat)
  ensures res == (n * (n + 1)) / 2
  // emphasizing how the expr. is parenthesized
{
  res := 0;
  var i := 1;
  while i <= n
    invariant i <= n + 1
    invariant res == ((i - 1) * i) / 2
  {
    calc {
      ((i - 1) * i) / 2;
    == (i * i - i) / 2;
    == (i * i + i - 2 * i) / 2;
    == (i * (i + 1) - 2 * i) / 2;
    == (i * (i + 1)) / 2 - (2 * i) / 2;
    == (i * (i + 1)) / 2 - i;
    }
    assert res + i == i * (i + 1) / 2;
    res := res + i;
    assert res == i * (i + 1) / 2;
    assert res == (i + 1 - 1) * (i + 1) / 2;
    i := i + 1;
    assert res == (i - 1) * i / 2;
  }
}
```

References: see Leino, chapter 11.